Reservas Deportivas

Guía de instalación

Paso a paso para que cada integrante del grupo pueda:

1. levantar la **API de la cátedra** en su computadora,
2. probar la app en el **emulador de Android Studio**,
3. instalarla en su **celular Android**, y
4. (opcional) **compilar** el APK o trabajar en el código.

> Todo está pensado para **Windows 10/11**. En macOS o Linux los pasos son los mismos; cambian los comandos de consola marcados como "Windows".

0. Qué se necesita

Para	Programa	Versión
Todo	**Git**	cualquiera reciente — https://git-scm.com
API y app	**Node.js**	**24 LTS** — https://nodejs.org
API	**pnpm** (viene con Node vía `corepack`)	10
Emulador / compilar	**Android Studio** (trae el SDK, `adb`, el emulador y Java 17)	la última estable
Editar código (opcional)	VS Code	—

Espacio en disco: unos **15 GB** (Android Studio + una imagen del emulador + dependencias).

1. Bajar los dos repositorios

La app y la API están en repositorios distintos. Conviene ponerlos en una carpeta **corta y sin espacios** (por ejemplo `C:\tfi`), porque la compilación de Android falla con rutas largas en Windows.

```
mkdir C:\tfi
cd C:\tfi
git clone <URL-de-este-repositorio> rd
git clone https://github.com/franco-ruisdias-uner/api-proyectos-mobile.git
```

Este repositorio se clona con el nombre corto `rd` para que la ruta quede corta. Quedan así:

```
C:\tfi\rd                                          ← este repositorio
C:\tfi\rd\reservas-deportivas                      ← la app
C:\tfi\api-proyectos-mobile\api-reservas-deportivas ← la API de la cátedra
```

2. Levantar la API en la PC

La app no tiene datos propios: todo viene de la API. La API corre en la computadora (modo local, permitido por la cátedra) y el emulador o el celular se conectan a ella.

2.1 Primera vez

Abrí una consola (PowerShell) en la carpeta de la API:

```
cd C:\tfi\api-proyectos-mobile\api-reservas-deportivas
corepack enable
pnpm install
copy .env.example .env
pnpm db:reiniciar
```

- `corepack enable` habilita `pnpm` (si da error de permisos, abrí PowerShell **como administrador**).
- En macOS / Linux, `cp .env.example .env` en lugar de `copy`.
- `pnpm db:reiniciar` crea la base y carga los **datos de demo** (espacios, usuarios, reservas).

2.2 Cada vez que se va a probar

```
cd C:\tfi\api-proyectos-mobile\api-reservas-deportivas
pnpm dev
```

Dejá esa ventana **abierta** mientras uses la app. Para comprobar que anda, abrí en el navegador http://localhost:3000/salud: tiene que responder algo como `{"datos":{"estado":"ok", ...}}`.

Ojo en Windows: si hacés clic dentro de la ventana de PowerShell, el título cambia a "Seleccionar…" y el programa queda **en pausa**. Apretá **Esc** para que siga.

Alternativa con Docker: si tenés Docker Desktop, en la carpeta de la API alcanza con `docker compose up`.

2.3 Volver los datos al estado inicial

Las pruebas crean reservas, bloqueos y fijas. Para empezar de cero: cortá `pnpm dev` (Ctrl+C) y corré `pnpm db:reiniciar`. Los datos de demo se calculan **a partir del día en que se cargan**, así que conviene reiniciar la base el mismo día de la prueba o de la defensa.

2.4 Verificar la API desde la app (opcional)

En la carpeta de la app:

```
cd C:\tfi\rd\reservas-deportivas
node scripts/probar-api.mjs http://localhost:3000
```

Tiene que terminar con **"Todo OK: la app puede usar esta API"**.

---

3. Probar en el emulador de Android Studio

3.1 Crear el emulador (una sola vez)

1. Instalá Android Studio y abrilo. En el asistente inicial dejá la instalación **Standard** (descarga el SDK, el emulador y las herramientas).
2. En la pantalla de bienvenida: **More Actions › Virtual Device Manager** (o **Device Manager**).
3. **Create device** (botón **+**) › elegí **Pixel 7** (o cualquier Pixel) › **Next**.
4. Imagen del sistema: **API 35 (Android 15)**, variante **x86_64 con Google APIs** › descargala › **Next** › **Finish**.
5. Tocá ▶ para arrancarlo. La primera vez tarda un par de minutos.

Si el emulador no arranca o va muy lento, falta la aceleración por hardware:

- Activá la virtualización (**VT-x / AMD-V / SVM**) en la BIOS.

- En Windows: *Activar o desactivar las características de Windows* › tildá **Plataforma del hipervisor de Windows** › reiniciá.
- Para comprobarlo (en una consola):

```
"%LOCALAPPDATA%\Android\Sdk\emulator\emulator.exe" -accel-check
```

Tiene que decir que la aceleración está disponible (WHPX o AEHD).

3.2 Poner el emulador en hora de Argentina (importante)

El emulador viene en hora **UTC** (3 horas adelantado). La API valida el ingreso con QR según la hora, y con UTC rechaza ingresos válidos ("Todavía no es el horario...").

En el emulador: **Settings › System › Date & time** › desactivá *Set time zone automatically* › **Time zone** › *Argentina* (Buenos Aires).

3.3 Instalar el APK en el emulador

El APK **no está dentro del repositorio** (pesa más de 100 MB, el límite de GitHub). Bajalo de la sección **Releases** de este repositorio en GitHub (archivo `ReservasDeportivas.apk`), o compilalo vos (sección 5).

Con el emulador abierto, cualquiera de estas dos formas:

- **Arrastrar** el archivo `.apk` y soltarlo encima de la ventana del emulador.
- Desde una consola:

```
"%LOCALAPPDATA%\Android\Sdk\platform-tools\adb.exe" install -r ReservasDeportivas.apk
```

El ícono es un calendario blanco con una pelota sobre fondo violeta: **Reservas Deportivas**.

3.4 Conectar la app con la API

1. Con la API corriendo (`pnpm dev`), abrí la app.
2. Pestaña **Perfil** › bajá hasta **Servidor**.
3. Escribí `http://10.0.2.2:3000` (en el emulador, `10.0.2.2` es tu PC; `localhost` sería el propio emulador).
4. Tocá **Probar** → tiene que decir *"Conectado. API 0.1.0, base better-sqlite3"*.
5. Tocá **Guardar** y volvé a **Espacios**: aparecen los 11 espacios.

3.5 Ubicación en el emulador (para el mapa)

En la barra lateral del emulador: □ **(Extended controls) › Location** › buscá *Villaguay, Entre Ríos* o escribí las coordenadas **-31.8635, -59.0262** › **Set location**. Así el mapa muestra tu punto y el complejo más cercano.

3.6 Huella en el emulador (opcional)

Para probar el reingreso con huella: en el emulador, **Settings › Security › Fingerprint** (requiere poner un PIN). Cuando pida apoyar el dedo, en **Extended controls › Fingerprint** tocá **Touch the sensor**. Sin huella, la app pide la contraseña (también es válido).

---

4. Instalar en un celular Android

Requisitos: Android 7.0 o más nuevo, unos 400 MB libres.

4.1 Pasar el APK al celular

- **Opción A:** abrí desde el celular la página de **Releases** del repositorio y descargá el `.apk`.
- **Opción B:** cable USB › en el celular elegí **Transferencia de archivos** › copiá el `.apk` a la carpeta **Download**.
- **Opción C:** Google Drive o Telegram (WhatsApp a veces bloquea los `.apk` grandes).

4.2 Instalarlo

1. Abrí el `.apk` desde la app **Archivos** (o desde la notificación de descarga).
2. Android avisa que no puede instalar apps de esa fuente: **Configuración › Permitir de esta fuente** › volvé atrás.
3. **Instalar**. Si aparece **Google Play Protect**: **Más detalles › Instalar de todas formas** (pasa porque la app no viene de Play Store).
4. **Abrir**.

> **"App no instalada":** desinstalá una versión anterior de la app (puede tener otra firma) y volvé a descargar el archivo completo.

4.3 Conectar el celular con la API de tu PC

Elegí una de estas dos formas y cargá la dirección en **Perfil › Servidor › Probar › Guardar**:

Forma	Dirección en la app	Qué hace falta
Wi-Fi (misma red que la PC)	`http://<IP-de-la-PC>:3000`, por ejemplo `http://192.168.0.10:3000`	La IP sale de `ipconfig` ("Dirección IPv4"). Permitir el puerto 3000 en el firewall (ver abajo).
Cable USB	`http://localhost:3000`	Activar **Depuración USB** en el celular y en la PC correr `adb reverse tcp:3000 tcp:3000` (repetirlo cada vez que se reconecta el cable).

Firewall de Windows (solo para Wi-Fi), en PowerShell **como administrador**, una sola vez:

```
netsh advfirewall firewall add rule name="API reservas 3000" dir=in action=allow protocol=TCP localport=3000 pro
```

La red Wi-Fi de la PC tiene que estar como **Red privada**. Si aun así no conecta (redes de facultad o públicas suelen aislar los equipos), usá el **hotspot del celular** o el **cable con** `adb reverse`.

Activar la depuración USB: Ajustes › Acerca del teléfono › tocá 7 veces **Número de compilación** (en Xiaomi: "Versión de MIUI/HyperOS") › volvé a Ajustes › **Opciones de desarrollador › Depuración USB**.

4.4 Permisos y batería según la marca

La app programa un recordatorio **2 horas antes** de cada turno. Varias marcas cierran las apps en segundo plano y cortan esas notificaciones. Mantené apretado el ícono › **Información de la app** y revisá:

Marca	Batería	Extra
Samsung	Batería › **Sin restricciones**	Si no deja instalar: desactivar temporalmente **Bloqueador automático** (Seguridad y privacidad).
Xiaomi / POCO / Redmi	Ahorro de batería › **Sin restricciones**	Activar **Autoinicio**.
Motorola / Pixel / Nokia	Batería › **Sin restricciones**	—

Marca	Batería	Extra
OPPO / Realme / OnePlus	Uso de batería › Permitir actividad en segundo plano	Activar **Inicio automático**.
Huawei / Honor	Inicio de aplicaciones › desactivar "Gestionar automáticamente"	—

Permisos que pide la app: **notificaciones** (al ingresar), **ubicación** (en el Mapa), **cámara** (solo el encargado, al escanear), **fotos** (solo el personal). Si se niega alguno, el resto de la app sigue funcionando.

---

5. Compilar el APK (opcional)

Solo hace falta si cambiaste código o no tenés el APK.

5.1 Preparar la app

```
cd C:\tfi\rd\reservas-deportivas
npm ci
npm test
npm run typecheck
npm run lint
```

`npm test` tiene que dar **38 pruebas OK**.

5.2 APK de prueba (local, con Android Studio instalado)

> **Windows:** la carpeta del proyecto tiene que tener una ruta **corta y sin espacios** (por ejemplo `C:\tfi\rd\reservas-deportivas`). Con rutas largas la compilación de C++ falla.

```
npx expo prebuild --platform android --clean
cd android
gradlew assembleRelease
```

El APK queda en `android\app\build\outputs\apk\release\app-release.apk` (firmado con la clave de prueba; sirve para instalar, no para Play Store). La carpeta `android/` se genera sola y **no se sube** al repo.

Si Gradle no encuentra el SDK, creá `android\local.properties` con:

```
sdk.dir=C:\\Users\\<tu-usuario>\\AppData\\Local\\Android\\Sdk
```

5.3 Modo desarrollo (cambios en vivo)

Con el emulador abierto (o un celular por cable):

```
npx expo run:android
```

Compila una versión de desarrollo, la instala y deja corriendo **Metro**: cada vez que guardás un archivo, la app se actualiza. **Expo Go no sirve** para esta app porque usa módulos nativos (MapLibre, SQLite...).

5.4 APK con EAS (en la nube, para la entrega)

Con la cuenta de Expo del grupo:

```
npx eas-cli@latest login
npx eas-cli@latest build --platform android --profile preview
```

Al terminar, EAS da un link para descargar el APK.

---

6. Usuarios de prueba

Todos con la contraseña `villaguay2026` (en la pantalla Ingresar hay botones de acceso rápido):

Email	Rol	Sirve para
`emiliano@mail.com`	Jugador	Reservas próximas y pasadas, Cancha 2 favorita
`lleno@demo.ar`	Jugador	Ya tiene 3 reservas activas (probar el límite)
`nuevo@demo.ar`	Jugador	Cuenta vacía para reservar desde cero
`marcelo@villaguay.demo.ar`	Encargado	Agenda, escanear, ausentes, bloqueos
`nahuel@villaguay.demo.ar`	Administrador	Lo del encargado + fijas y precios

---

7. Problemas frecuentes

Problema	Solución
"Falta configurar la dirección del servidor"	Perfil › Servidor › escribir la dirección › Probar › Guardar.
"No hay conexión con el servidor"	¿Está corriendo `pnpm dev`? Emulador: `10.0.2.2`, no `localhost`. Wi-Fi: misma red, IP correcta, firewall. Cable: repetir `adb reverse tcp:3000 tcp:3000`.
Se queda cargando / "El servidor tardó demasiado"	La ventana de `pnpm dev` está en pausa ("Seleccionar"): apretar Esc. O la dirección está mal escrita.
"Todavía no es el horario de esa reserva" en el emulador	Poner el emulador en hora de Argentina (sección 3.2).
"Tenés que ingresar con tu cuenta" de golpe	Se reinició la base de la API: volver a ingresar.
El mapa se ve gris	El mapa (OpenStreetMap) necesita internet; la API local no alcanza.
El emulador no arranca o va lentísimo	Falta la aceleración (sección 3.1).
`pnpm` no se reconoce	Correr `corepack enable` (como administrador si hace falta) y abrir una consola nueva.
La compilación falla con errores de rutas / `ninja`	Mover el proyecto a una ruta corta sin espacios (`C:\tfi\...`).
No llegan notificaciones	Permitir notificaciones y batería "Sin restricciones" (sección 4.4).
"App no instalada"	Desinstalar la versión anterior y volver a descargar el APK.